

Report 2 for Task 2: Architecture

- Andrei Foitoş (S5233836)
- Andrei-George Iclodean (S6480039)
- Yuwen Zhou (S5521351)

Executive Summary

This report presents the architecture and deployment pipeline for a machine learning system designed to classify and process review data. The system implements a microservices architecture using Docker containers, orchestrated through Docker Compose, with a comprehensive CI/CD pipeline managed by GitLab CI/CD. The architecture supports asynchronous model inference, centralized model management via MLflow, and comprehensive monitoring through Prometheus, Loki, and Grafana.

System Architecture

To describe our ADD Detection System architecture at multiple levels of abstraction, we adopt the C4 model, which refines the system from its external context to its internal structure. It enables a clear separation of concerns and supports reasoning about system responsibilities, deployment boundaries and interactions between components.

We focus only on the first three levels of the C4 model. Level 4 focuses on code-level structures such as classes and methods, which are highly implementation-specific. Since the objective of this assignment is to reason about system architecture, deployment, asynchronous processing, and observability, Levels 1 to 3 already provide sufficient abstraction without introducing unnecessary coupling to implementation details.

The system follows a microservices architecture pattern, implementing the C4 model for architectural documentation. The architecture is designed to support scalable machine learning inference through asynchronous task processing, model versioning and experimentation via MLflow, data persistence across PostgreSQL and MongoDB, a comprehensive observability through metrics, logs, and dashboards, and containerized deployment for consistency across environments.

Level 1: System Context Diagram

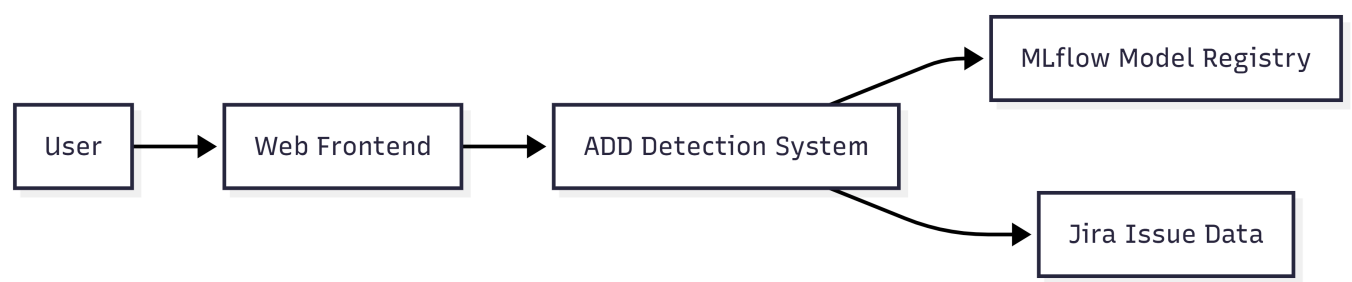


Figure 1: System context diagram

Figure 1 is the System Context Diagram that illustrates how the ADD Detection System is situated within its environment and how it interacts with external actors and systems. At this level, the system is treated as a

single black box. Users interact with the system through a web-based frontend to submit Jira issues, request batch predictions, and perform keyword-based searches for Architectural Design Decisions (ADDs). The ADD Detection System relies on two primary external systems: the MLflow Model Registry, which provides access to versioned machine learning models, and Jira Issue Data, which serves as the source of issue descriptions and summaries to be analyzed.

Level 2: Container Diagram

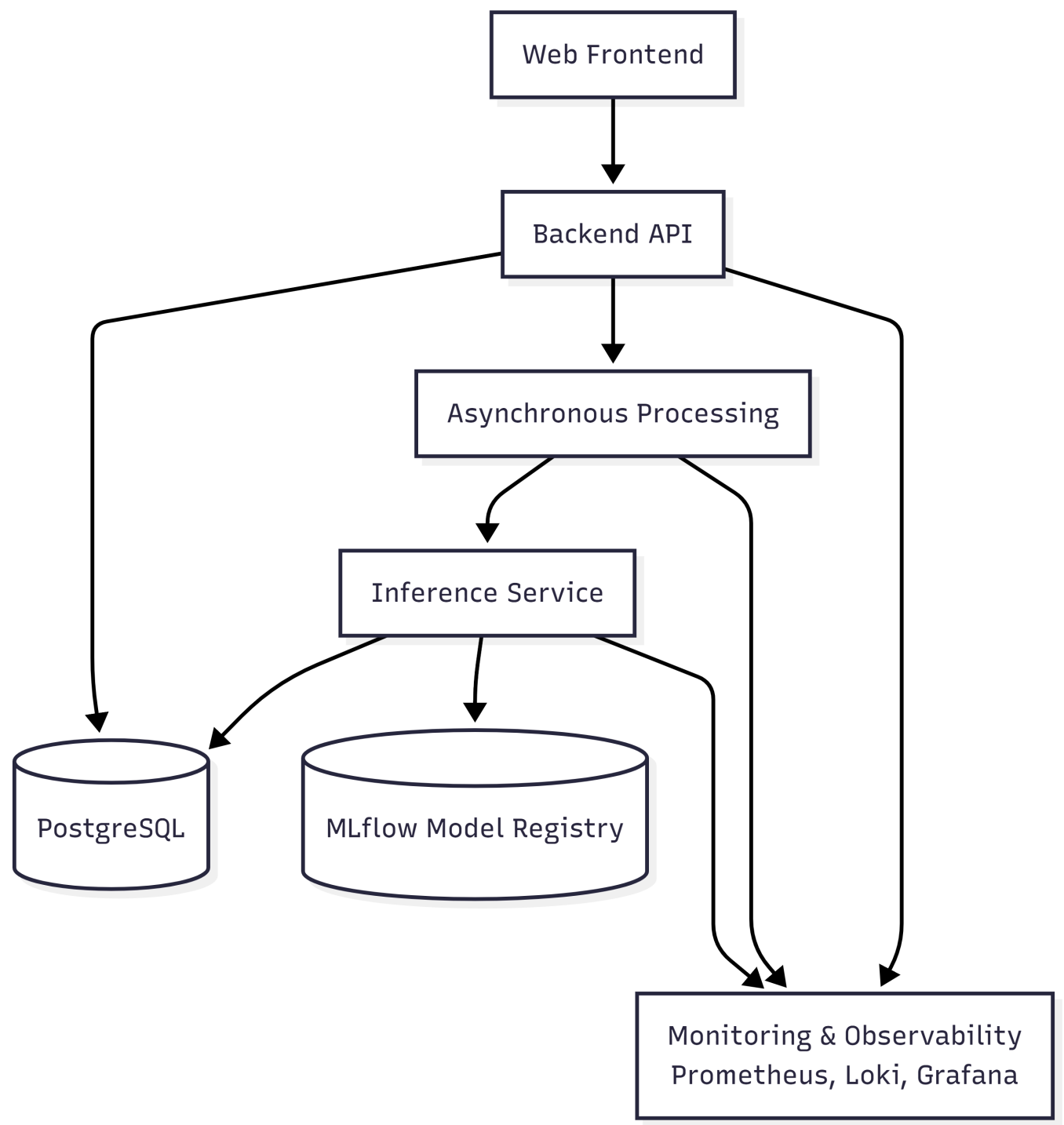


Figure 2: Container Diagram

Figure 2 showcases the Container Diagram, which decomposes the ADD detection system into its main deployable units and shows how responsibilities are distributed across them. The system consists of five primary containers:

- **Backend API:** It acts as the central entry point for all client requests. It exposes REST endpoints for single prediction, batch submission, status queries and keyword-based search.
- **Asynchronous Processing:** It handles long-running and computationally intensive tasks, such as batch inference, outside the request-response cycle to ensure API responsiveness.
- **Inference Service:** It executes machine learning inference by loading the model from MLflow Model Registry and storing prediction results.
- **PostgreSQL:** It stores issues, prediction results, batch task metadata and supports keyword- based search.
- **Monitoring Stack:** It collects metrics and logs from backend services and provides a unified observability dashboard.

When a user submits a request via the frontend, it is forwarded to the Backend API. Lightweight operations (e.g., single prediction or search) are handled directly, while batch requests are delegated to the asynchronous processing component. The inference service retrieves the trained model from MLflow, performs predictions, and persists results to the database. Metrics and logs generated by the backend and inference services are continuously collected by the monitoring stack, enabling system monitoring and debugging.

Deployed Infrastructure Context

Although Figure 2 abstracts deployment details, all containers shown in the diagram are deployed on a single virtual machine and orchestrated using Docker Compose. Each container (API, Celery workers, Inference service, PostgreSQL, Redis, MLflow, MinIO, Prometheus, Loki, and Grafana) runs as an isolated Docker service within the same internal Docker network. Only the Backend API and Grafana expose ports to the host machine, while all other services are accessible exclusively via the internal network.

This deployment model ensures clear service boundaries, controlled network access, and reproducible infrastructure across environments while remaining consistent with the Container Diagram abstraction.

Level 3: Component Diagram

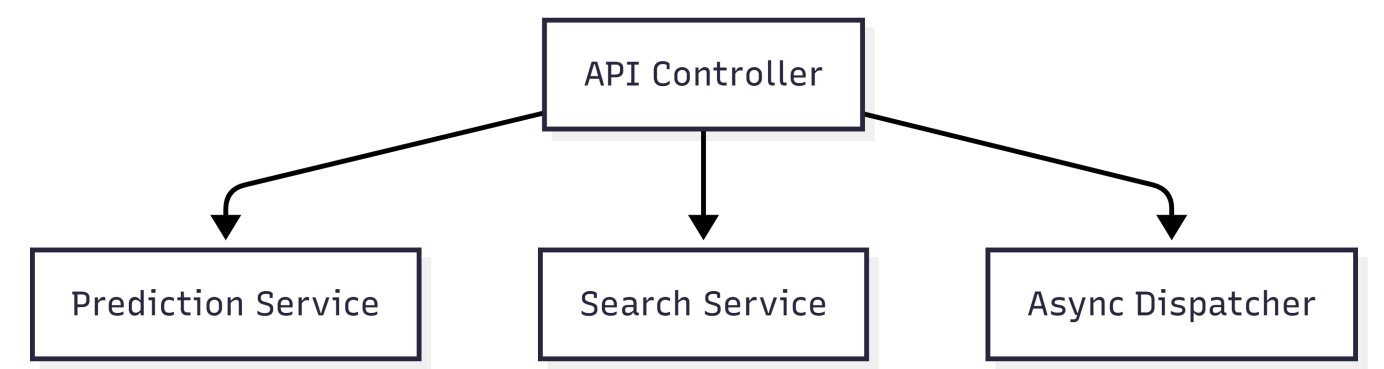


Figure 3: Component Diagram

The component Diagram shown in Figure 3, focuses on the internal structure of the Backend API container and highlights the key components that implement the system’s functionality. The Backend API is composed of the following logical components:

- **API Controller:** It defines REST endpoints and handles incoming HTTP requests from the web frontend.

- Prediction Service: It manages synchronous prediction requests and communicates with the inference service.
- Search Service: It handles keyword-based retrieval of ADD-related issues using the database.
- Async Dispatcher: It submits batch predictions to the asynchronous processing mechanism and enables non-blocking request handling.

Once received a request, the API Controller routes it to the appropriate internal service. Batch prediction requests are forwarded to the Async Dispatcher, which decouples long-running inference tasks from the API's request-handling path. Throughout this process, metrics and logs are generated to support monitoring and debugging.

API Responsibilities as Reflected in the Architecture

Figure 3 clarifies the functional responsibilities of the Backend API by decomposing it into components that directly correspond to its externally exposed endpoints. The API Controller represents the REST interface responsible for request validation and routing, while the Prediction Service, Search Service, and Async Dispatcher encapsulate the core business logic behind synchronous predictions, keyword-based search, and batch processing respectively.

This separation of concerns ensures that the API serves both as a stable interface for clients and as a coordination layer that delegates computation-heavy tasks to downstream services without blocking client requests.

Component Interactions

API Purpose and Functionality

The FastAPI application serves as the primary interface for the ML system, providing Architectural Decision Documentation (ADD) detection for software project issues.

Core Endpoints:

```
POST /predictions          # Submit issue for ADD classification
GET  /predictions/{task_id} # Retrieve classification results
GET  /hello                # Health check endpoint
GET  /metrics               # Prometheus metrics (auto-exposed)
```

Functionality Flow:

1. Request Reception: User submits issue via POST /predictions with summary and description
2. Input Validation: Pydantic automatically validates request schema
3. Task Submission: API creates Celery task tasks.classify_issue with issue text
4. Immediate Response: Returns HTTP 202 Accepted with task_id (non-blocking)
5. Result Polling: User queries GET /predictions/{task_id} to check status
6. Result Delivery: Once complete, returns ADD classification with confidence score

Response Example:

```
{
  "task_id": "a7f3c2e1-4b5d-6789-0abc-def123456789",
  "status": "SUCCESS",
  "result": {
    "is_add": true,
    "probability": 0.8734,
    "label": "ADD"
  }
}
```

API-Model Interaction in the Architectural Diagrams

As illustrated in Figures 2 and 3, the Backend API does not directly embed the machine learning model but instead interacts with it through a dedicated inference workflow. The Prediction Service delegates inference requests to the inference execution logic, which dynamically loads the currently active model version from the MLflow Model Registry.

This architectural separation allows the API to remain model-agnostic, enabling model updates, rollbacks, and experimentation without requiring changes to the API layer itself. The diagrams thus highlight MLflow as an external dependency responsible for model lifecycle management rather than a tightly coupled internal component.

Asynchronous Processing Implementation

Asynchronous Processing in the Architectural View

The asynchronous processing mechanism is explicitly reflected in the Container and Component diagrams. The Async Dispatcher component shown in Figure 3 represents the architectural boundary between request handling and long-running computation. In Figure 2, this decoupling is reinforced by the presence of a message broker (Redis) and separate worker containers responsible for task execution.

By visualizing asynchronous task submission and worker-based execution as independent components, the diagrams emphasize how non-blocking request handling is achieved and how the system can scale horizontally by increasing the number of Celery workers without modifying the API.

The system implements asynchronous processing using the Celery distributed task queue with Redis as the message broker.

Architecture Benefits:

1. Non-blocking API: Users receive immediate responses with task IDs
2. Scalability: Multiple Celery workers can process tasks in parallel
3. Reliability: Redis persists tasks, enabling retry mechanisms
4. Resource efficiency: Long-running ML inference doesn't block API threads

Task Flow:

1. FastAPI receives prediction request
2. Creates Celery task with review text
3. Task pushed to Redis queue
4. Available Celery worker picks up task
5. Worker performs inference using loaded model
6. Result stored in PostgreSQL with task ID
7. User retrieves result via task ID

Frontend and API Interaction

The current system architecture does not include a dedicated frontend application. The API is designed to be consumed by direct HTTP clients and future frontend applications.

Client Interaction Perspective

In the System Context Diagram (Figure 1), the frontend is represented abstractly as an external user or HTTP client interacting with the system through RESTful API calls. While no dedicated frontend application is currently deployed, the diagram intentionally models this interaction to capture how any future web interface, CLI tool, or third-party system would communicate with the Backend API.

This abstraction ensures that client-API interaction patterns are documented independently of frontend implementation details, aligning with the C4 model's emphasis on stable system boundaries.

Integration of Prometheus, Loki, and Grafana

Observability Components in the Architectural Diagrams

The Container Diagram (Figure 2) groups Prometheus, Loki, and Grafana into a unified monitoring stack to emphasize their collective role in system observability. Prometheus is responsible for scraping metrics exposed by the Backend API and worker services, Loki aggregates logs emitted by all containers, and Grafana acts as the visualization layer that correlates metrics and logs.

Although represented as a logical group, each component operates independently and communicates through well-defined interfaces, enabling metrics-based monitoring, centralized log analysis, and cross-component observability across the entire system.

The monitoring stack provides comprehensive observability across all system components.

Component Roles:

Prometheus (Port 9090):

- Scrapes metrics from instrumented applications
- Stores time-series data with labels
- Configured via prometheus/prometheus.yml
- Metrics collected:
 - API request rates (requests/second)

- Response latencies (p50, p95, p99)
- Error rates (4xx, 5xx responses)
- Task queue lengths
- Model inference duration

Loki (Port 3100):

- Aggregates logs from Docker containers
- Uses Docker's JSON file logging driver
- Each container tagged with service name
- Indexes logs by timestamp and labels
- Enables full-text log search

Grafana (Port 3000):

- Unified dashboard for metrics and logs
- Pre-configured data sources:
 - Prometheus for metrics visualization
 - Loki for log exploration
- Default credentials: admin/admin
- Provisioned via grafana/provisioning/ directory

Data Flow:

1. FastAPI exports metrics via prometheus-fastapi-instrumentator
2. Prometheus scrapes metrics endpoint every 15 seconds
3. Docker containers write logs to stdout/stderr
4. Loki collects logs via Docker logging driver
5. Grafana queries both data sources for unified view

Monitoring Strategy

Logs

Log Sources:

- API logs: Request/response details, errors, warnings
- Celery worker logs: Task execution, model loading, inference times
- MLflow logs: Model registry operations, artifact storage
- Database logs: Connection issues, slow queries

Tracked Metrics

API Performance Metrics:

Metric	Type	Description	Relevance
http_requests_total	Counter	Total HTTP requests	Track API usage patterns
http_request_duration_seconds	Histogram	Request latency distribution	Identify performance bottlenecks
http_requests_in_progress	Gauge	Concurrent requests	Monitor load and capacity
http_response_status	Counter	Response codes (2xx, 4xx, 5xx)	Track error rates

ML System Metrics:

Metric	Type	Description	Relevance
celery_tasks_total	Counter	Total tasks submitted	Monitor inference volume
celery_task_duration_seconds	Histogram	Task processing time	Detect model performance issues
celery_worker_active_tasks	Gauge	Tasks in progress	Monitor worker utilization
model_inference_duration_seconds	Histogram	Pure model inference time	Track model efficiency
model_load_duration_seconds	Histogram	Model loading time	Optimize model caching

Infrastructure Metrics:

Metric	Type	Description	Relevance
redis_connected_clients	Gauge	Active Redis connections	Monitor queue health
postgres_active_connections	Gauge	Database connections	Detect connection leaks
container_memory_usage_bytes	Gauge	Memory consumption	Prevent OOM errors
container_cpu_usage_percent	Gauge	CPU utilization	Right-size container resources

Alerting Rules

Critical Alerts:

- API error rate > 5% for 5 minutes
- Average response time > 2 seconds for 10 minutes
- Celery worker count = 0
- Database connection failures

Warning Alerts:

- Queue length > 100 tasks

- Model inference time > 5 seconds
- Memory usage > 80%

Pipeline Architecture

CI/CD Strategy Overview

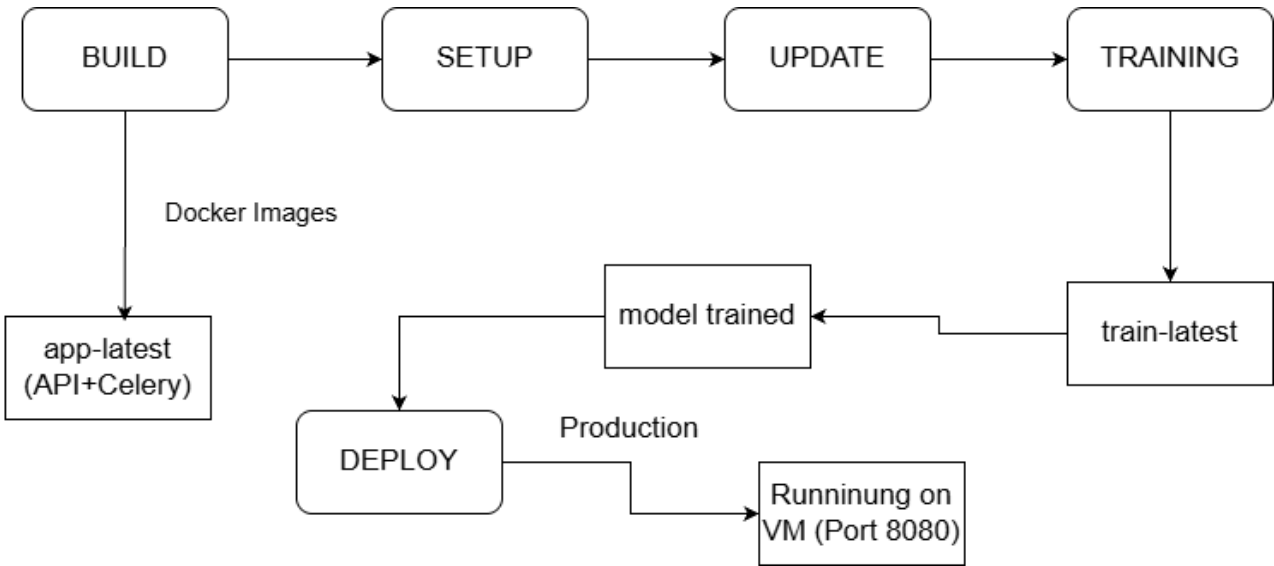
The CI/CD pipeline is implemented using GitLab CI/CD, orchestrating automated build, test, and deployment processes. The strategy follows a multi-stage pipeline approach with conditional execution based on code changes.

CI/CD Pipeline Flow Description

The CI/CD pipeline follows a linear, multi-stage execution model that progresses from image building to deployment. Conceptually, the pipeline starts with the BUILD stage, where application and training images are created and pushed to the container registry. This is followed by SETUP, which ensures that all infrastructure services are running on the target VM.

Subsequent stages manage data ingestion and preprocessing (UPDATE), model training and registration (TRAINING), and finally API deployment (DEPLOY). Conditional execution rules ensure that only the relevant stages are triggered based on changes in code, data, or configuration files, minimizing unnecessary pipeline executions while maintaining deployment consistency.

Pipeline Diagram:



Key Principles:

1. Infrastructure as Code: All deployment configurations in version control
2. Automated Testing: Validation at each stage (currently minimal, room for improvement)
3. Conditional Execution: Stages trigger only when relevant files change
4. Container-based Deployment: Consistent environments from dev to production
5. Centralized Secrets: Environment variables managed via GitLab CI/CD settings

Pipeline Stages

Stage 1: BUILD

Purpose: Build and publish Docker images to GitLab Container Registry

Jobs:

build_app (API + Celery Application)

```
build_app:
  stage: build
  image: docker:20.10.16
  services:
    - docker:20.10.16-dind
  script:
    - docker build -t "$CI_REGISTRY_IMAGE:app-latest" -f backend/api/Dockerfile .
    - docker push "$CI_REGISTRY_IMAGE:app-latest"
  rules:
    - changes:
        - backend/api/**
        - backend/inference/**
        - environment.yml
```

- Trigger: Changes to API code, inference code, or dependencies
- Dockerfile: backend/api/Dockerfile
- Output: Image tagged as app-latest in registry
- Contents: FastAPI application, Celery tasks, Python dependencies

build_train_image (Training Pipeline)

```
build_train_image:
  stage: build
  script:
    - docker build -t "$CI_REGISTRY_IMAGE:train-latest" -f
      backend/inference/Dockerfile .
    - docker push "$CI_REGISTRY_IMAGE:train-latest"
  rules:
    - changes:
        - data/training/**
        - backend/inference/**
        - environment.yml
```

- Trigger: Changes to training scripts, inference code, or dependencies
- Dockerfile: backend/inference/Dockerfile
- Output: Image tagged as train-latest
- Contents: Training scripts, model code, PyTorch, transformers

Security in Build Stage:

- Uses GitLab's built-in CI_REGISTRY_PASSWORD and CI_REGISTRY_USER
- Credentials never hardcoded in .gitlab-ci.yml
- Docker-in-Docker (DinD) for isolated builds

Stage 2: SETUP

Purpose: Deploy infrastructure and services to target VM

Job:

setup_infra

```
setup_infra:
  stage: setup
  script:
    - rsync -rtvz --delete --exclude 'volumes' . "$CI_SSH_HOST":/home/$VM_USER/app
    - ssh $CI_SSH_HOST "cd /home/$VM_USER/app && sudo docker compose down && sudo
docker compose up -d"
  rules:
    - when: always
```

Actions:

1. Sync files: rsync transfers docker-compose.yml, configs to VM
2. Recreate services: Stops and restarts all Docker Compose services
3. Initialize infrastructure:
 - PostgreSQL databases (reviews_db, MLflow metadata)
 - Redis task queue
 - MLflow tracking server
 - MinIO object storage
 - Monitoring stack (Prometheus, Loki, Grafana)

Stage 3: UPDATE (Data Management)

Purpose: Load and preprocess data into the system

Job: update_data

```
update_data:
  stage: update
  script:
    - ssh $CI_SSH_HOST "docker run --name ppl-load_data \
      --network mlsto-assignment_assignment \
```

```

    -v /home/$VM_USER/app/data:/data \
    -e POSTGRES_URL='$POSTGRES_URL' \
    -e MLFLOW_TRACKING_URL='$MLFLOW_TRACKING_URL' \
    -e MLFLOW_MODEL_NAME='$MLFLOW_MODEL_NAME' \
    $CI_REGISTRY_IMAGE:app-latest \
    python /app/data/preprocessing/ppl-load_data.py"
rules:
  - changes:
    - data/preprocessing/**
    - backend/inference/**

```

Data Management Script Functionality:

The system implements a three-stage data pipeline tracked by DVC:

Stage 1: Data Extraction

```

# Connects to MongoDB containing JIRA issues and labels
client = MongoClient("mongodb://root:my-secret-pw@localhost:27017")
labels_db = client["MiningDesignDecisions"]
jira_db = client["JiraRepos"]

# Query labeled issues
cursor = labels_db.find({
    "tags": "has-label",
    "existence": {"$exists": True},
    "property": {"$exists": True},
    "executive": {"$exists": True}
})

# Join with JIRA issue data
for label in cursor:
    project, issue_id = label_id.split("-", 1)
    issue = jira_db[project].find_one({"id": issue_id})

    # Clean HTML tags and whitespace
    description = clean_text(fields.get("description", ""))
    summary = clean_text(fields.get("summary", ""))

    # Collect labeled records
    records.append({
        "project": project,
        "label_id": label_id,
        "description": description,
        "summary": summary,
        "existence": bool(label.get("existence", False)),
        "property": bool(label.get("property", False)),
        "executive": bool(label.get("executive", False))
    })

```

```
# Export to CSV
df.to_csv("data/issue_with_labels.csv", index=False)
```

Key Features:

- Extracts issues from MongoDB databases (MiningDesignDecisions + JiraRepos)
- Joins labels with JIRA issue content
- Cleans HTML tags using regex: TAG_RE = re.compile(r"<[^>]+>")
- Filters empty issues (both summary and description blank)
- Outputs labeled dataset with three label types: existence, property, executive

Stage 2: Data Validation (pandera_check.py)

```
schema = DataFrameSchema({
    "project": Column(pa.String, nullable=False),
    "label_id": Column(pa.String, nullable=False),
    "summary": Column(pa.String, Check.str_length(min_value=0)),
    "description": Column(pa.String, Check.str_length(min_value=0)),
    "existence": Column(pa.Bool),
    "property": Column(pa.Bool),
    "executive": Column(pa.Bool)
}),
checks=Check(
    lambda df: ~((df["summary"].str.len() == 0) &
                  (df["description"].str.len() == 0)),
    error="summary and description both empty"
))

schema.validate(df)
```

Validation Rules:

- Schema enforcement: correct data types for all columns
- Non-null constraints on project and label_id
- String length validation on text fields
- Data quality check: prevents records with both empty summary and description
- Ensures boolean labels (existence, property, executive) are properly typed

Stage 3: Database Loading (ppl-load_data.py)

```
def load_csv_to_postgres(csv_file_path, db_config):
    df = pd.read_csv(csv_file_path)

    # Create table schema
    cursor.execute("""
CREATE TABLE processed_issues (
```

```

        issue_key VARCHAR(50) PRIMARY KEY,
        summary TEXT,
        description TEXT,
        label_existence BOOLEAN,
        label_executive BOOLEAN,
        label_property BOOLEAN,
        project VARCHAR(50)
    )
    """

# Bulk insert using psycopg2 extras
data_tuples = [
    (row.label_id, row.summary, row.description,
     row.existence, row.executive, row.property, row.project)
    for row in df.itertuples(index=False)
]

extras.execute_values(cursor, query, data_tuples)

```

Database Schema:

- Table: processed_issues
- Primary Key: issue_key (e.g., "Apache-13343357")
- Text Fields: summary, description (unlimited length)
- Labels: Three boolean fields for ADD classification types
- Metadata: Project identifier for tracking source
- Bulk Loading: Uses psycopg2.extras.execute_values() for efficient batch inserts

DVC Integration:

```

# dvc.yaml
stages:
  preprocess:
    cmd: python data/preprocessing/preprocessing.py
    deps:
      - data/preprocessing/preprocessing.py
    outs:
      - data/issue_with_labels.csv

```

- Preprocessed data is versioned via DVC (hash: 5537ca73561f385c05b6bb3f02202555)
- Enables data versioning parallel to code versioning
- .dvc/cache stores historical data versions

Execution Pattern:

- Runs as one-off container (--name ppl-load_data)
- Mounts data directory for file access
- Connects to internal Docker network

- Container removed after completion

Stage 4: TRAINING (Model Training)

Purpose: Train ML models using versioned data

Job: train_model

```
train_model:
  stage: training
  script:
    - ssh $CI_SSH_HOST "docker run -d --name ppl-train \
      --network mlsdo-assignment_assignment \
      -e POSTGRES_URL='$POSTGRES_URL' \
      -e REDIS_URL='$REDIS_URL' \
      -e MLFLOW_TRACKING_URL='$MLFLOW_TRACKING_URL' \
      -e MLFLOW_MODEL_NAME='$MLFLOW_MODEL_NAME' \
      $CI_REGISTRY_IMAGE:train-latest"
  rules:
    - changes:
      - data/training/**
      - backend/inference/**
      - environment.yml
```

Model Training Script Functionality:

1. Data retrieval: Fetches training data from PostgreSQL
2. Model training: Fine-tunes transformer model (e.g., BERT, DistilBERT)
3. Hyperparameter tracking: Logs parameters (learning rate, batch size) to MLflow
4. Metrics logging: Records training/validation accuracy, F1-score, loss
5. Model registration: Saves trained model to MLflow Model Registry
6. Artifact storage: Stores model weights to MinIO via MLflow

MLflow Integration:

```
# Pseudocode from training script
import mlflow

mlflow.set_tracking_uri(os.getenv('MLFLOW_TRACKING_URI'))
with mlflow.start_run():
    mlflow.log_params({"learning_rate": 2e-5, "epochs": 3})

    # Train model
    model.train()

    mlflow.log_metrics({"accuracy": 0.92, "f1_score": 0.89})
    mlflow.pytorch.log_model(model, "model")
    mlflow.register_model(f"runs:{run_id}/model", "ReviewClassifier")
```

Execution Characteristics:

- Runs detached (-d) for long-running training jobs
- Connects to Redis for potential distributed training coordination
- Outputs logged to Docker container logs (accessible via Loki)
- Limitation: No explicit completion notification in pipeline

Model Versioning:

- Each training run creates a new MLflow experiment run
- Models tagged with version numbers (v1, v2, etc.)
- Production models promoted via MLflow Model Registry stages (Staging → Production)
- Enables A/B testing and rollback capabilities

Stage 5: DEPLOY

Purpose: Deploy the trained API service to production

Job: deploy_api

```
deploy_api:
  stage: deploy
  script:
    - ssh $CI_SSH_HOST "docker pull $CI_REGISTRY_IMAGE:app-latest"
    - ssh $CI_SSH_HOST "docker stop app-api || true"
    - ssh $CI_SSH_HOST "docker rm app-api || true"
    - ssh $CI_SSH_HOST "docker run -d --name app-api \
      --network mlsdo-assignment_assignment \
      -e POSTGRES_URL='$POSTGRES_URL' \
      -e REDIS_URL='$REDIS_URL' \
      -e MLFLOW_TRACKING_URL='$MLFLOW_TRACKING_URL' \
      -e MLFLOW_MODEL_NAME='$MLFLOW_MODEL_NAME' \
      -p 8080:8080 \
      $CI_REGISTRY_IMAGE:app-latest"
  rules:
    - changes:
        - backend/api/**
        - backend/inference/**
        - environment.yml
```

Deployment Process:

1. Pull latest image: Ensures most recent build is available
2. Graceful shutdown: Stops existing app-api container
3. Clean up: Removes old container
4. Deploy new version: Starts fresh container with updated code
5. Port mapping: Exposes port 8080 to host

6. Network attachment: Connects to internal Docker network

Security Measures

Secret Management

GitLab CI/CD Variables: All sensitive credentials are stored as protected and masked CI/CD variables in GitLab:

Variable	Purpose	Scope
POSTGRES_URL	Database connection string	Protected, Masked
REDIS_URL	Task queue connection	Protected, Masked
MLFLOW_TRACKING_URL	MLflow server endpoint	Protected
MLFLOW_MODEL_NAME	Model registry identifier	Protected
CI_REGISTRY_PASSWORD	Container registry authentication	Built-in, Masked
CI_SSH_HOST	Deployment target VM	Protected

Security Properties:

- Protected: Only available on protected branches (main, production)
- Masked: Values hidden in job logs
- Never committed: Credentials never appear in .gitlab-ci.yml or code

SSH Access Control

Authentication:

```
image: finalgene/openssh
```

- Pipeline uses SSH for remote command execution
- SSH keys configured via GitLab CI/CD SSH keys settings
- Private key stored securely in GitLab
- Public key added to VM's authorized_keys

VM User Configuration:

- Dedicated user warpgate with limited sudo permissions
- Sudo access restricted to Docker commands only
- No password-based authentication

Container Registry Security

GitLab Container Registry:

- Built-in authentication via GitLab tokens

- gitlab-ci-token with job-specific permissions
- Images scoped to project (not publicly accessible)
- Automatic token rotation

Build Stage Authentication:

```
before_script:
  - echo "$CI_REGISTRY_PASSWORD" | docker login $CI_REGISTRY -u $CI_REGISTRY_USER
  --password-stdin
```

Deployment Stage Authentication:

```
before_script:
  - ssh $CI_SSH_HOST "docker login -u gitlab-ci-token -p $CI_JOB_TOKEN
    $CI_REGISTRY"
```

Network Isolation

Docker Network Security:

- Internal network mlsdo-assignment_assignment isolates services
- Only API container exposes external port (8080)
- Database and Redis not accessible from outside
- Service-to-service communication within Docker network

Production Hardening Recommendations:

1. Database credentials: Move from docker-compose.yml to environment variables

```
# Current (insecure)
POSTGRES_PASSWORD: pw1

# Recommended
POSTGRES_PASSWORD: ${POSTGRES_PASSWORD} # From .env file or secrets
```

1. TLS/SSL: Enable HTTPS for API endpoints
2. Firewall rules: Restrict VM access to specific IP ranges
3. Secrets rotation: Implement periodic credential rotation
4. Image scanning: Add vulnerability scanning to build stage

```
stage: build
script:
  - trivy image $CI_REGISTRY_IMAGE:app-latest
```

Dependency Management

Supply Chain Security:

1. Pinned versions: environment.yml specifies exact versions

```
- torch==2.2.0  
- celery==5.6.2  
- redis==4.5.5
```

1. Trusted sources: Only PyPI and conda-forge channels
2. Docker base images: Official Python images with specific tags

Access Control

Role-Based Access:

- GitLab project permissions control who can trigger pipelines
- Protected branches prevent unauthorized deployments
- Manual approval gates could be added for production deployments

Audit Trail:

- All pipeline executions logged in GitLab
- Git commit history provides change tracking
- MLflow tracks model lineage and experiment metadata